

Circuiti e porte logiche

Definizioni utili

Iniziamo a fare qualcosa che più si avvicina all'idea che probabilmente avevamo di Architettura degli Elaboratori: oggi ci occupiamo dei circuiti e delle porte logiche! Prima di tutto diamo qualche definizione utile, così da chiamare ogni cosa con il suo nome.

Un **circuito**, in elettronica, è l'insieme di elementi connessi tra loro in modo che nell'insieme circoli, o possa circolare, una corrente elettrica.

Un **circuito logico** è un circuito che attua una determinata funzione logica, primitiva, direttamente derivata o complessa; è formato da una o più porte logiche.

Una **porta logica**, o *gate*, è un circuito in grado di implementare un'operazione logica primitiva (AND, OR, NOT) o direttamente derivata (NAND, NOR, XOR).

Un **circuito integrato** è la realizzazione fisica di un circuito logico, miniaturizzato, dove tutti i componenti vengono costruiti con procedimenti chimici su una piastra solitamente di silicio; le dimensioni sono così ottimizzate. Si distinguono per **livello di integrazione**:

- SSI (Small Scale Integrated): 1-10 porte
- MSI (Medium Scale Integrated): 10-100 porte
- LSI (Large Scale Integrated): 100-100.000 porte
- VLSI (Very Large Scale Integrated): > 100.000 porte

Ora che abbiamo un'idea più chiara di quello di cui stiamo parlando, entriamo nella questione dei segnali. Come abbiamo accennato all'[inizio < https://blue3141.dev/knowledge-base/codifica-dellinformazione/>](https://blue3141.dev/knowledge-base/codifica-dellinformazione/), il computer riesce a leggere solo 2 stati: acceso o spento, alto o basso, corrente sì o corrente no, per intenderci. Il problema è che fisicamente è improbabile avere tale precisione nei segnali, ossia avere esattamente la massima tensione o esattamente 0 V. Per questo si discretizza il segnale, e si considera il segnale alto se ha voltaggio superiore a 1 V, mentre basso se inferiore o uguale.

Un **segnale** viene definito **alto** e viene indicato con **1** se ha voltaggio > 1 V; altrimenti viene definito **basso** e viene indicato con **0**.

Valgono i principi della logica che abbiamo imparato a Fondamenti: identità (il segnale, a meno di variazioni nel tempo, rimane tale), non contraddittorietà (un segnale non può essere sia alto che basso contemporaneamente) e del terzo escluso (se non è alto, è per forza basso e viceversa).

Ora vediamo come distinguere i circuiti in base a cosa dipende il risultato: infatti, se di fronte a input istantanei identici l'output è lo stesso, e il risultato non dipende quindi da valori salvati in memoria derivati da operazioni precedenti, si parlerà di circuito combinatorio; se invece il circuito ha al suo interno dei circuiti adibiti al salvataggio di determinati stati, che vengono poi usati per calcolare l'output, si tratta di un circuito sequenziale.

Si parla di **circuito combinatorio** o **rete combinatoria** se l'output dipende solo dalla funzione logica applicata allo stato istantaneo, ossia se non usa dei dati salvati in memoria.

Si parla di **circuito sequenziale** o **rete sequenziale** se l'output dipende anche da stati precedentemente salvati all'interno del circuito.

Bene, abbiamo finito con le definizioni!

Porte logiche

Porte primitive

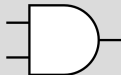
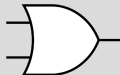
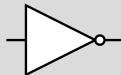
				
a	b	Porta AND	Porta OR	Porta NOT
0	0	0	0	1
0	1	0	1	1
1	0	0	1	0
1	1	1	1	0

Tabella delle porte logiche primitive. La porta NOT usa solo l'input a.

Come possiamo vedere, le porte implementano le funzioni logiche che abbiamo già visto a Fondamenti; la porta NOT è chiamata anche inverter, in quanto inverte il segnale in input. Se serve negare il segnale subito prima di farlo entrare in una porta (non NOT), si usa un pallino tra l'input e la porta!

Porte logiche derivate

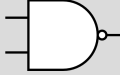
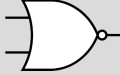
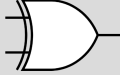
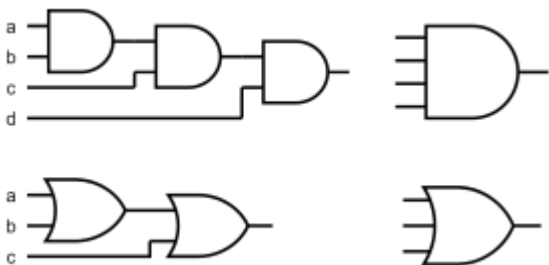
				
a	b	Porta NAND	Porta NOR	Porta XOR
0	0	1	1	0
0	1	1	0	1
1	0	1	0	1
1	1	0	0	0

Tabella delle porte logiche derivate.

Ecco le porte logiche derivate: è semplice riconoscerle, il pallino indica un segnale negato! L'ultima porta, l'OR disgiuntivo (*aut aut* latino) si riconosce per la doppia linea alla base.

Porte con più ingressi

Cosa succede quando abbiamo più di due input, per esempio vogliamo calcolare $a \text{ AND } b \text{ AND } c$? La risposta è semplice: facciamo una “cascata” di porte! Certo è scomodo disegnarle, così per i diagrammi si mettono semplicemente più entrate alla porta. Qui sotto vediamo una porta AND a 4 input e una OR a 3 input, sia come cascata che come rappresentazione semplificata.



< <https://i0.wp.com/blue3141.dev/wp-content/uploads/2021/07/image-2.png?ssl=1> >

Esempi di porte logiche a più input: cascata e relativa rappresentazione semplificata.

Universalità di NAND e NOR

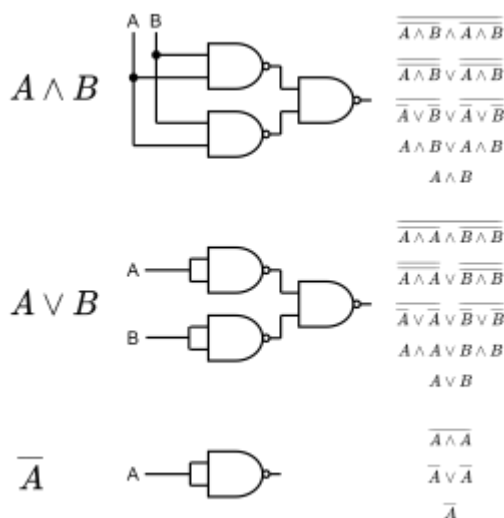
Come abbiamo detto prima, usando le porte AND, OR e NOT si può implementare qualsiasi funzione logica. In verità, per rappresentarle ci basterebbero solo porte NAND, oppure solo porte NOR. Sono dette quindi porte universali! Dimostriamo che possiamo ricavare le porte AND, OR e NOT da queste due porte.

Universalità di NAND

AND: $(A \text{ NAND } B) \text{ NAND } (A \text{ NAND } B)$

OR: $(A \text{ NAND } A) \text{ NAND } (B \text{ NAND } B)$

NOT : $A \text{ NAND } A$



< <https://i0.wp.com/blue3141.dev/wp-content/uploads/2021/07/image-3.png?ssl=1> >

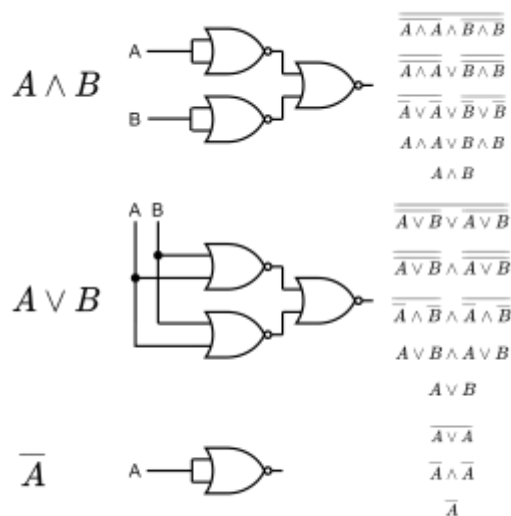
Universalità di NAND.

Universalità di NOR

AND: $(A \text{ NOR } A) \text{ NOR } (B \text{ NOR } B)$

OR: $(A \text{ NOR } B) \text{ NOR } (A \text{ NOR } B)$

NOT : $B \text{ NOR } B$



< <https://i0.wp.com/blue3141.dev/wp-content/uploads/2021/07/image-4.png?ssl=1> >

Universalità di NOR.

Parallelismo fra operazioni logiche e matematiche

Possiamo anche pensare alle operazioni logiche come operazioni matematiche binarie (senza tener conto del resto): l'AND è la moltiplicazione (se un fattore è 0 il risultato è 0), l'OR è l'addizione (se qualcosa è 1, il risultato è 1). Potete controllare con le tabelle logiche sopra! Ci sarebbe anche il NOT, che è $1-X$ (inverte il valore: $1-1=0$ e $1-0=1$), ma solitamente non si “traduce” in operazione, bensì si tiene la barra sopra. Così possiamo scrivere, anziché $\overline{A \wedge B \vee C}$, questo: $\overline{A \cdot B + C}$. Ci sarà utile fra poco!

Circuiti logici notevoli

Decoder

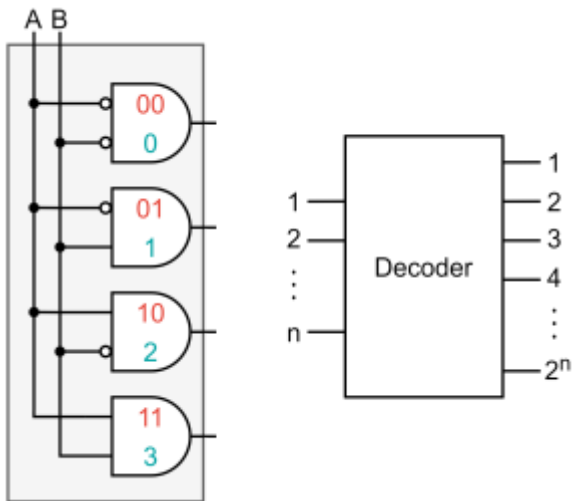
Il decoder è un circuito che ha n entrate, e 2^n uscite: ad ogni set di input possibile in input viene associata la relativa (ed unica) uscita. Possiamo vederlo come un “convertitore” da binario a decimale: in input riceviamo il numero in binario, ossia n bit, e viene accesa la k -esima uscita, dove k è la conversione in base 10 del numero in entrata. Vediamo com'è costruito!

Decoder

n input

2^n output

Output: è alta solamente la k -esima porta dove k è la conversione in base dieci dell'input n .



[< https://i0.wp.com/blue3141.dev/wp-content/uploads/2021/07/image-5.png?ssl=1 >](https://i0.wp.com/blue3141.dev/wp-content/uploads/2021/07/image-5.png?ssl=1)

Architettura interna di un decoder e relativa rappresentazione.

Abbiamo un decoder a 2 entrate (numero binario AB) e quindi a 4 uscite. Le porte AND sono realizzate negando determinati segnali: prendiamo la porta numero 0 (in verde), deve restituire vero se in entrata abbiamo 00 (in rosso), quindi nega entrambi in modo che 00 diventi 11 e quindi 1 AND 1. Stesso discorso per gli altri numeri: per input 10, per esempio, si deve accendere la porta 2; quindi questa porta negherà il segnale di B, così 10 diventa 1 AND 1 e restituisce vero. Guardare la tabella per credere!

A	B	Porta 0	Porta 1	Porta 2	Porta 3
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1

Tabella di verità del decoder a 2 entrate e 4 uscite.

Multiplexor

Abbreviato Mux, il multiplexor (o multiplexer) ha 2^n input principali, più altre n porte di selezione in input, e restituisce un solo output, quello corrispondente all'input selezionato dal selettore (o dai selettori). Si può pensare ad una tabella dove ci sono 2^n valori di verità in posizione 0, 1, 2, 3...il selettore sarà un numero binario che ci dirà in quale posizione andare per prendere il valore da restituire!

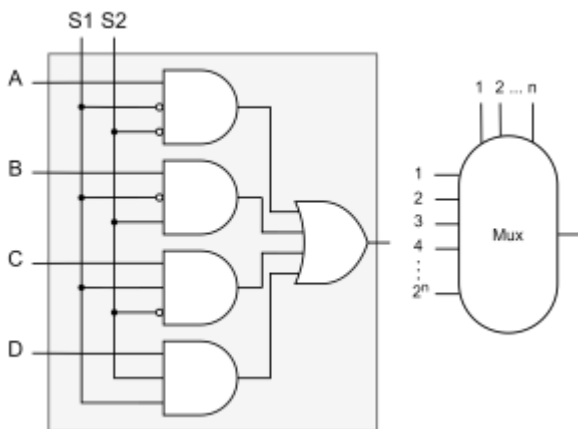
Multiplexor

2^n (valori) + n (selettori) input

1 output

Output: il valore del dato selezionato dai selettori, ossia il valore del k -esimo dato dove k è la conversione in base 10 del numero binario formato dai valori dei selettori.

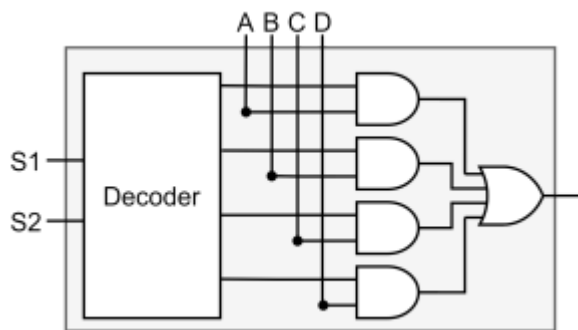
Ecco la rappresentazione grafica:



<https://i0.wp.com/blue3141.dev/wp-content/uploads/2021/07/image-6.png?ssl=1>

Architettura interna di un mux e relativa rappresentazione.

Dopo aver visto il decoder è semplice capire il mux: i segnali selettori “accendono” una delle porte AND esattamente come fa il decoder; quindi l'unico segnale X fra quelli a sinistra che potrà passare sarà quello di quella porta, in quanto fa $X \text{ AND } 1$. Dopodiché il segnale viene messo in una porta OR con tutti gli altri (che sono sicuramente 0, perché i selettori non le hanno “attivate”): quindi se il segnale selezionato è 0 il risultato è 0, altrimenti 1. Come previsto! Possiamo semplificare la rappresentazione inserendo un decoder:



< <https://i0.wp.com/blue3141.dev/wp-content/uploads/2021/07/image-7.png?ssl=1>

Multiplexor rappresentato con un decoder.

Ed ora vediamo la tabella di verità: un trattino significa che può essere scelto qualsiasi valore di verità, perché tanto la porta AND relativa a quell'input (non selezionato) ha 0 come operando grazie ai selettori.

S1	S2	Porta AND selezionata	A	B	C	D	Output
0	0	0 (A)	0	–	–	–	0
0	0	0 (A)	1	–	–	–	1
0	1	1 (B)	–	0	–	–	0
0	1	1 (B)	–	1	–	–	1
1	0	2 (C)	–	–	0	–	0
1	0	2 (C)	–	–	1	–	1
1	1	3 (D)	–	–	–	0	0
1	1	3 (D)	–	–	–	1	1

Tabella di verità del decoder a 2 entrate e 4 uscite.

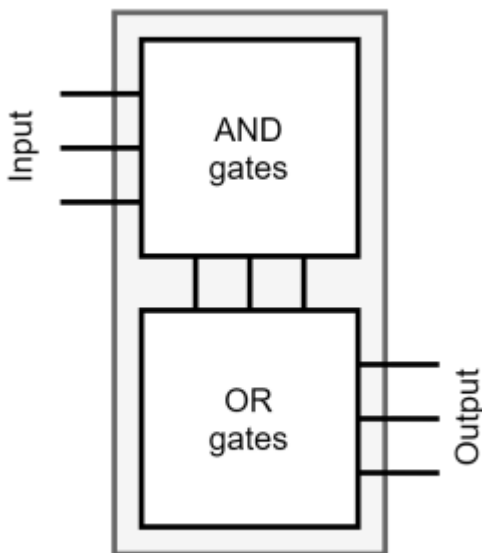
Si tratta di un componente che avremo modo di usare non poche volte, soprattutto più avanti, quindi è meglio capire bene come funziona!

Logiche a due livelli: PLA

Abbiamo detto che con AND, OR e NOT possiamo implementare qualsiasi funzione logica. In particolare, possiamo farlo in due passi: prima con delle porte AND e poi OR, o viceversa. Ricordando che l'AND corrisponde al prodotto e l'OR all'addizione, possiamo dire che possiamo rappresentare le funzioni come somme di prodotti (SOP, *sum of products*) o come prodotti di somme (POS, *product of sums*). So che può sembrare complicato, ma nella pratica poi è semplice! Noi vedremo nello specifico quando un circuito è implementato secondo somme di prodotti, e parliamo quindi di un PLA.

Un **PLA** (Programmable Logic Array) è un circuito in cui gli input vengono elaborati prima da un array di porte AND (primo stadio, prodotto) e poi da un array di porte OR (secondo stadio, somma).

Vediamo la rappresentazione grafica:



< <https://i0.wp.com/blue3141.dev/wp-content/uploads/2021/07/image-8.png?ssl=1> >

Rappresentazione grafica di un PLA.

Tutto fantastico, ma nella pratica come si fa a calcolare la somma di prodotti relativa ad una certa funzione logica? I passi sono 3!

Ricavare la somma di prodotti da una tabella di verità

- 1) Prendere tutti i set di valori di verità degli input che restituiscono come output 1 (i modelli, per chiamarli con la terminologia di Fondamenti).
- 2) Per ogni set, se il valore di un input è 0, negarlo. Scrivere il prodotto di tutti gli

input (negati e non) per ogni set.

3) Sommare tutti i prodotti.

Esempio 1

Proviamo a ricavare la SOP della seguente tabella di verità con 3 input e 2 output:

I ₁	I ₂	I ₃	O ₁	O ₂
0	0	0	1	1
0	0	1	0	0
0	1	0	0	1
0	1	1	0	1
1	0	0	1	1
1	0	1	0	1
1	1	0	1	0
1	1	1	1	1

Iniziamo a calcolare la SOP per il primo output. Prendiamo i set di verità che restituiscono $O_1 = 1$:

I ₁	I ₂	I ₃	O ₁	O ₂
0	0	0	\circled{1}	1
0	0	1	0	0
0	1	0	0	1
0	1	1	0	1
1	0	0	\circled{1}	1
1	0	1	0	1
1	1	0	\circled{1}	0
1	1	1	\circled{1}	1

Quindi i modelli di O_1 sono

$$\{I_1, I_2, I_3\} = \{0, 0, 0\}$$

$$\{I_1, I_2, I_3\} = \{1, 0, 0\}$$

$$\{I_1, I_2, I_3\} = \{1, 1, 0\}$$

$$\{I_1, I_2, I_3\} = \{1, 1, 1\}$$

Ora dobbiamo negare gli input uguali a 0, e li moltiplichiamo:

$$\{0, 0, 0\} \rightarrow \{\bar{I}_1, \bar{I}_2, \bar{I}_3\} \rightarrow \bar{I}_1 \cdot \bar{I}_2 \cdot \bar{I}_3$$

$$\{1, 0, 0\} \rightarrow \{I_1, \overline{I_2}, \overline{I_3}\} \rightarrow I_1 \cdot \overline{I_2} \cdot \overline{I_3}$$

$$\{1, 1, 0\} \rightarrow \{I_1, I_2, \overline{I_3}\} \rightarrow I_1 \cdot I_2 \cdot \overline{I_3}$$

$$\{1, 1, 1\} \rightarrow \{I_1, I_2, I_3\} \rightarrow I_1 \cdot I_2 \cdot I_3$$

E ora li sommiamo tutti!

$$O_1 = (\overline{I_1} \cdot \overline{I_2} \cdot \overline{I_3}) + (I_1 \cdot \overline{I_2} \cdot \overline{I_3}) + (I_1 \cdot I_2 \cdot \overline{I_3}) + (I_1 \cdot I_2 \cdot I_3)$$

Abbiamo finito per il primo input! Se notate, avendo negato gli input uguali a 0, ci assicuriamo che quando arriva un modello la moltiplicazione (porta OR) restituisca 1, quindi la somma dà sicuramente 1.

Per il secondo input, vediamo che ci sono tanti 1 e pochi 0...quindi, perché non negare l'output? L'idea è questa: anziché ricavare O_2 , ricaviamo $\overline{O_2}$ e poi neghiamo alla fine tutta la somma di prodotti ricavata, tanto le porte NOT le possiamo mettere dove vogliamo! Riscriviamo quindi la tabella:

I_1	I_2	I_3	O_1	$\overline{O_2}$
0	0	0	1	0
0	0	1	0	\circled1
0	1	0	0	0
0	1	1	0	0
1	0	0	1	0
1	0	1	0	0
1	1	0	1	\circled1
1	1	1	1	0

$$\{I_1, I_2, I_3\} = \{0, 0, 1\}$$

$$\{I_1, I_2, I_3\} = \{1, 1, 0\}$$

$$\{0, 0, 1\} \rightarrow \{\overline{I_1}, \overline{I_2}, I_3\} \rightarrow \overline{I_1} \cdot \overline{I_2} \cdot I_3$$

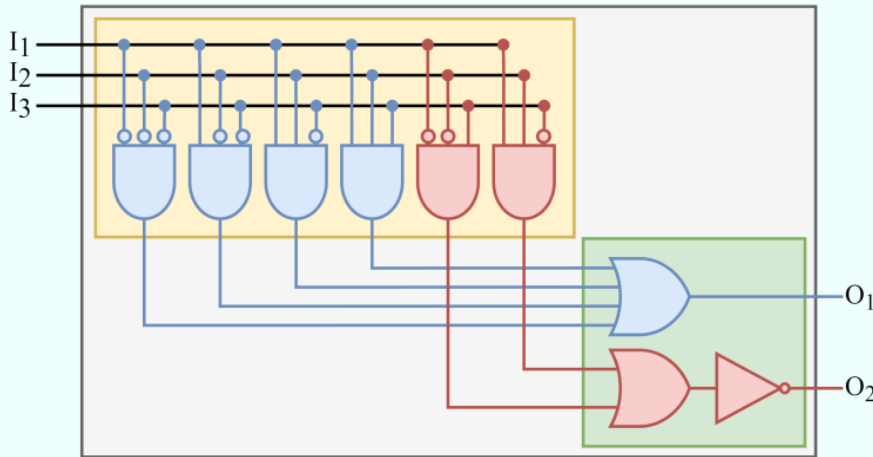
$$\{1, 1, 0\} \rightarrow \{I_1, I_2, \overline{I_3}\} \rightarrow I_1 \cdot I_2 \cdot \overline{I_3}$$

$$\overline{O_2} = (\overline{I_1} \cdot \overline{I_2} \cdot I_3) + (I_1 \cdot I_2 \cdot \overline{I_3})$$

$$O_2 = \overline{(\overline{I_1} \cdot \overline{I_2} \cdot I_3) + (I_1 \cdot I_2 \cdot \overline{I_3})}$$

Decisamente più compatto, abbiamo solo due termini!

Ora facciamo il PLA: per ogni output, ogni gruppo di parentesi (...) equivale ad una porta AND. I risultati delle porte AND vanno in un'unica porta OR. Ne risulta che avremo tante porte AND quanti sono i modelli (nel secondo caso abbiamo preso i contromodelli, neghiamo tutto alla fine per farli diventare modelli! Quindi avremo (modelli del primo output + contromodelli del secondo output) porte AND) e tante porte OR quanti sono gli output. Ecco qui!



Vediamo nel riquadro giallo l'array di porte AND, mentre in quello verde le porte OR. Le porte azzurre sono per il primo output, le rosse per il secondo; l'inverter del secondo input (visto che abbiamo usato i contromodelli!) l'ho messo "per esteso" e non solo come pallino per renderlo ben visibile. Carino, no? 😊

ROM (Read Only Memory)

Per implementare una funzione, basterebbe semplicemente memorizzare il risultato che deve restituire per ogni combinazione di valori in entrata, no? Ecco che le ROM fanno il loro ingresso! Sono dei circuiti con delle locazioni di memoria; quando ricevono un input n , restituiscono come output il valore salvato nel posto numero k (dove k , di nuovo, è la conversione in base decimale di n).

Una **ROM** (Read Only Memory) è un circuito che include 2^n (altezza della memoria) locazioni di memoria a m bit (larghezza della memoria: ogni dato salvato ha m bit) accessibili attraverso un indirizzo a n bit. Il loro contenuto è fisso e viene indicato durante la fabbricazione.

Le differenze fra le PLA e le ROM sono queste:

PLA	ROM
Partially decoded (codifica solo una combinazione di modelli e contromodelli, un input può servire per più output)	Fully decoded (ha tutti i risultati, ad un input corrisponde un solo indirizzo e quindi valore)
Dimensioni minori	Dimensioni maggiori
Più efficiente	Meno efficiente
Per stesso numero di input e output, volendo cambiare funzione può cambiare la dimensione	Dato il numero di input e di output, la dimensione della ROM richiesta è la stessa qualsiasi sia la funzione

Tabella per il confronto fra PLA e ROM.

Esistono anche le PROM (Programmable ROM), che possono essere programmate una sola volta (ROM “personalizzabili”), le EPROM (Erasable PROM) che permettono la cancellazione dei valori attraverso la luce ultravioletta e la successiva riscrittura, e le EEPROM (Electrically Erasable PROM) la cui programmazione e cancellatura vengono gestite elettricamente.

E per ora abbiamo finito! Il prossimo argomento sarà la ALU 😊